Exercise: Matrix Multiplication Performance Comparison

Objective

Compare the runtime performance of matrix multiplication across three implementations:

- **Pure C** - Low-level compiled language
- **Pure Python** - High-level interpreted language with native loops
- **NumPy** - Python with optimized numerical library

This exercise demonstrates the performance tradeoffs between different programming approaches and shows why numerical computing libraries are essential for Python.

Exercise Requirements

Part 1: Implementation

1.1 C Implementation

Write a C program that:

- Implements matrix multiplication using three nested loops
- Accepts matrix size as a command-line argument
- Measures execution time using clock() or gettimeofday()
- Initializes matrices with random values between 0 and 1
- Prints the execution time in milliseconds

File: matrix_mult.c

Key requirements:

- Use dynamic memory allocation (malloc)
- Properly free allocated memory
- Compile with optimization flag: gcc -O3 matrix_mult.c -o matrix_mult

1.2 Pure Python Implementation

Write a Python function that:

- Implements matrix multiplication using three nested loops
- Uses built-in list data structures (no NumPy)
- Measures execution time using time.time() or timeit

- Initializes matrices with random values

1.3 NumPy Implementation

Write a Python function that:

- Uses NumPy's @ operator or np.matmul()
- Measures execution time
- Uses the same random initialization approach

1.4 Timing Framework

Create a systematic testing framework that:

- Tests each implementation with multiple matrix sizes
- Runs multiple trials (at least 3-5) for each size and takes the average
- Handles the compilation and execution of C code from Python
- Collects all timing data in a structured format (DataFrame)

Suggested matrix sizes: 64×64, 128×128, 256×256, 512×512, 1024×1024, 2048×2048

Note: For larger sizes (1024+), Pure Python may take a very long time. Consider timing out or using smaller sizes for Python.

Part 2: Jupyter Notebook Report

Create a comprehensive Jupyter notebook with the following sections:

2.1 Introduction

- Overview of the three approaches being compared
- Explain the timing methodology
- Describe how matrices are initialized
- List the matrix sizes tested

2.3 Implementation Code

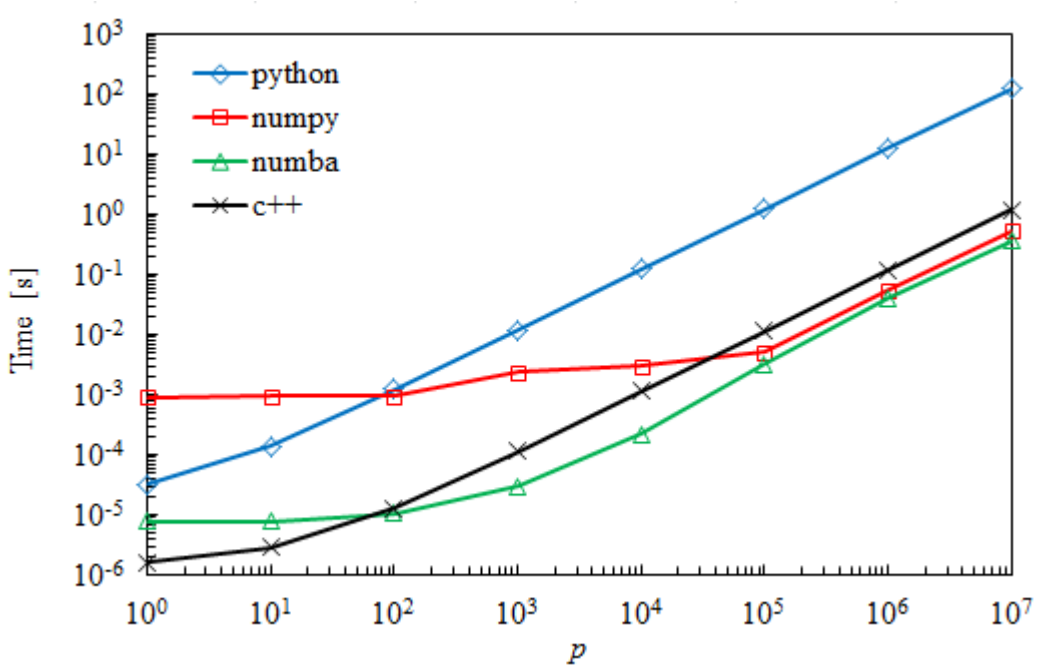
- Include or reference all three implementations
- Add comments explaining key parts of the code
- Show how the C code is compiled and executed from Python

2.4 Results and Visualization

Required visualizations:

1. **Line plot**: Execution time vs. matrix size for all three implementations
    - X-axis: Matrix size (n)
    - Y-axis: Time (milliseconds, log scale recommended)
    - Three lines for C, Python, and NumPy

    Example



2. **Speedup analysis**:
    - Create a plot showing speedup factors
    - NumPy speedup over Pure Python
    - C speedup over Pure Python
    - NumPy vs C comparison
3. **Data table**: Include a formatted table with all timing results

2.5 Analysis and Discussion

Answer the following questions with evidence from your data:

1. **Performance ranking**: Which implementation is fastest? Slowest? By how much?
2. **Scaling behavior**: How does execution time grow with matrix size for each implementation? Does it match $O(n^3)$ expectations?
3. **Python vs C**: Why is Pure Python so much slower than C?

4. **NumPy performance**:
    - Why is NumPy so much faster than Pure Python?
    - How does NumPy compare to C? Why?
    - What optimizations does NumPy use? (BLAS, LAPACK, vectorization, cache efficiency)
5. **Unexpected results**: Were there any surprising findings? Explain.

2.6 Conclusion

- Summarize key findings

Deliverables

1. **matrix_mult.c** - C implementation
2. **Jupyter Notebook (.ipynb)** - Complete report with:
    - All Python code (inline)
    - Timing results
    - Visualizations
    - Written analysis
3. **README.md** - Instructions for running the code